

Paws Rescue Donations

A Deep Dive into a Django Donation Tracker

Explainer for `paws-rescue-donations`

What you are about to read

This document explains every meaningful part of the `paws-rescue-donations` codebase, assuming you have never written a line of Python or seen a web application before. Every technical term is defined the first time it appears. The code, not the README, is the authority throughout: where the two disagree, that disagreement is reported as a finding rather than smoothed over.

Contents

1	The project in one page	3
1.1	The problem it solves	3
1.2	The shape of the whole thing	3
1.3	Scale and honesty about it	3
2	Django, briefly, for someone who has never met it	4
2.1	The directory layout	5
3	Architecture	6
3.1	The whole system at a glance	6
3.2	The two apps and what separates them	7
4	The accounts app: who is allowed in	7
4.1	Why there is a custom user model at all	7
4.2	<code>UserManager</code> : why dropping <code>username</code> forces this	8
4.3	The login path	10
4.4	Admin management, and where it is weaker than it looks	11
4.5	The <code>create_initial_admin</code> command	12
5	The donations app: the actual subject matter	13
5.1	The <code>Donation</code> model	13
5.2	Two forms, doing quite different jobs	14
5.3	<code>dashboard_view</code> : the heart of the application	15
5.4	CSV export, and the two bugs in it	17
5.5	The three CRUD views	19
6	Configuration: <code>settings.py</code>	19
6.1	Environment variables and the twelve-factor idea	19
6.2	The database switch	20
6.3	Static files and <code>Whitenoise</code>	20
6.4	Sessions	22
7	Templates and the front end	22
8	The mixed-currency bug	23

9	Testing	23
9.1	What is covered, and it genuinely passes	23
9.2	Where the suite is weaker than its pass rate suggests	24
10	Honest findings, collected	25
11	What is genuinely good here	26
12	If this project continued	27

1 The project in one page

1.1 The problem it solves

An animal rescue takes in money. Somebody donates 1000 rupees and earmarks it for a specific cat named Bella. Somebody else donates 500 for a dog named Max. A volunteer needs to answer, later, questions like “how much has Bella received this month” and “show me every bank transfer above 2000 rupees”.

The usual tool for this is a spreadsheet. A spreadsheet works until several people are entering records at once, at which point three problems arrive together: there is no access control (anyone with the link can edit or delete anything), there are no consistent fields (one person types “Bank”, another types “bank transfer”), and there is no reliable way to ask a question of the data.

This project replaces the spreadsheet with a small database-backed web application sitting behind a login. It was built for a faculty-supervised Khidmat community-service project at Habib University.

Jargon: Web application

A program that runs on a computer somewhere else (the *server*) and that you use through a web browser. The browser sends a *request* (“show me the dashboard”), the server does some work and sends back a *response* (a page of HTML that the browser draws on screen).

Jargon: Database

A program dedicated to storing structured records and answering questions about them quickly and safely, even when many users read and write at the same time. This project uses SQLite for local development and PostgreSQL in production. Both are *relational* databases: they store data in tables of rows and columns, like a spreadsheet, but with enforced rules about what may go in each column.

1.2 The shape of the whole thing

The application has exactly one kind of user: a staff member who logs in. There is no public-facing side at all, no donor-visible page, no sign-up form. Everything is behind the login.

The entire feature set

- Log in with an email address and password.
- Create, read, update and delete donation records.
- Filter and sort those records seven different ways.
- See running totals and a “top 5 pets” panel on a dashboard.
- Export the current filtered view as a CSV file.
- Add and remove other staff logins from the web interface.

1.3 Scale and honesty about it

This is a small project, and this document treats it as one. The entire application is roughly 900 lines of Python plus about 600 lines of HTML templates. There is no background job queue, no cache, no message broker, no microservice, and no JavaScript written by hand. That is a reasonable set of choices for the problem, and the document does not inflate them into an architecture they are not.

2 Django, briefly, for someone who has never met it

Every file in this project is arranged the way it is because Django expects it that way. Understanding the framework's vocabulary makes the rest of the document read easily.

Jargon: Framework

A pre-built skeleton of a program. Rather than writing the plumbing that listens for browser requests, matches URLs, talks to the database and renders HTML, you write only the parts specific to your problem and drop them into slots the framework provides. Django is a web framework for Python.

Jargon: MVT (Model, View, Template)

Django's way of splitting a web application into three jobs. **Model**: a Python class describing one kind of record, which Django maps onto a database table. **View**: a Python function that receives a request, decides what to do, and returns a response. **Template**: an HTML file with placeholders that get filled in with data. Django calls this MVT; other frameworks call the same idea MVC.

Jargon: ORM (Object-Relational Mapper)

The layer that lets you write `Donation.objects.filter(pet_name='Max')` in Python instead of writing `SELECT * FROM donations WHERE pet_name = 'Max'` in SQL. It translates Python method calls into database queries and turns the returned rows back into Python objects.

Jargon: QuerySet

The object the ORM hands back when you ask for records. Its defining property is *laziness*: it does not touch the database when you build it, only when you actually read from it. This matters enormously in this codebase, because it is what lets `dashboard_view` stack seven filters onto one QuerySet and still fire only one query at the end.

Jargon: Migration

A recorded, versioned instruction for changing the database's structure (adding a table, adding a column). Django generates these from your models and applies them in order, so that a fresh database and a long-lived one end up with identical structure.

2.1 The directory layout

```

paws-rescue-donations
├── manage.py the command-line entry point
├── Procfile tells the host how to start the app
├── requirements.txt the six Python dependencies
├── .env.example template for local configuration
├── db.sqlite3 a committed database, see §10
├── paws_rescue/ project configuration, not features
│   ├── settings.py every configuration decision
│   ├── urls.py the top-level URL map
│   └── wsgi.py the handle a web server grabs
├── apps/ the two feature apps
│   ├── accounts/ who may log in
│   │   ├── models.py
│   │   ├── forms.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── management/commands/create_initial_admin.py
│   │   ├── migrations/0001_initial.py
│   │   ├── templates/accounts/
│   │   └── tests.py
│   └── donations/ the actual subject matter
│       ├── models.py
│       ├── forms.py
│       ├── views.py
│       ├── urls.py
│       ├── migrations/0001_initial.py
│       ├── templates/donations/
│       └── tests.py
├── templates/base.html the page frame every page extends
└── static/css/custom.css

```

Figure 1: The tracked files, with the noise (empty `__init__.py` files, screenshots) omitted.

Jargon: Django “app”

A confusing name. An app is not a separate program; it is a folder grouping the models, views and templates for one area of responsibility. One Django *project* contains many apps. Here the project is `paws_rescue` and it contains two apps, `accounts` and `donations`.

3 Architecture

3.1 The whole system at a glance

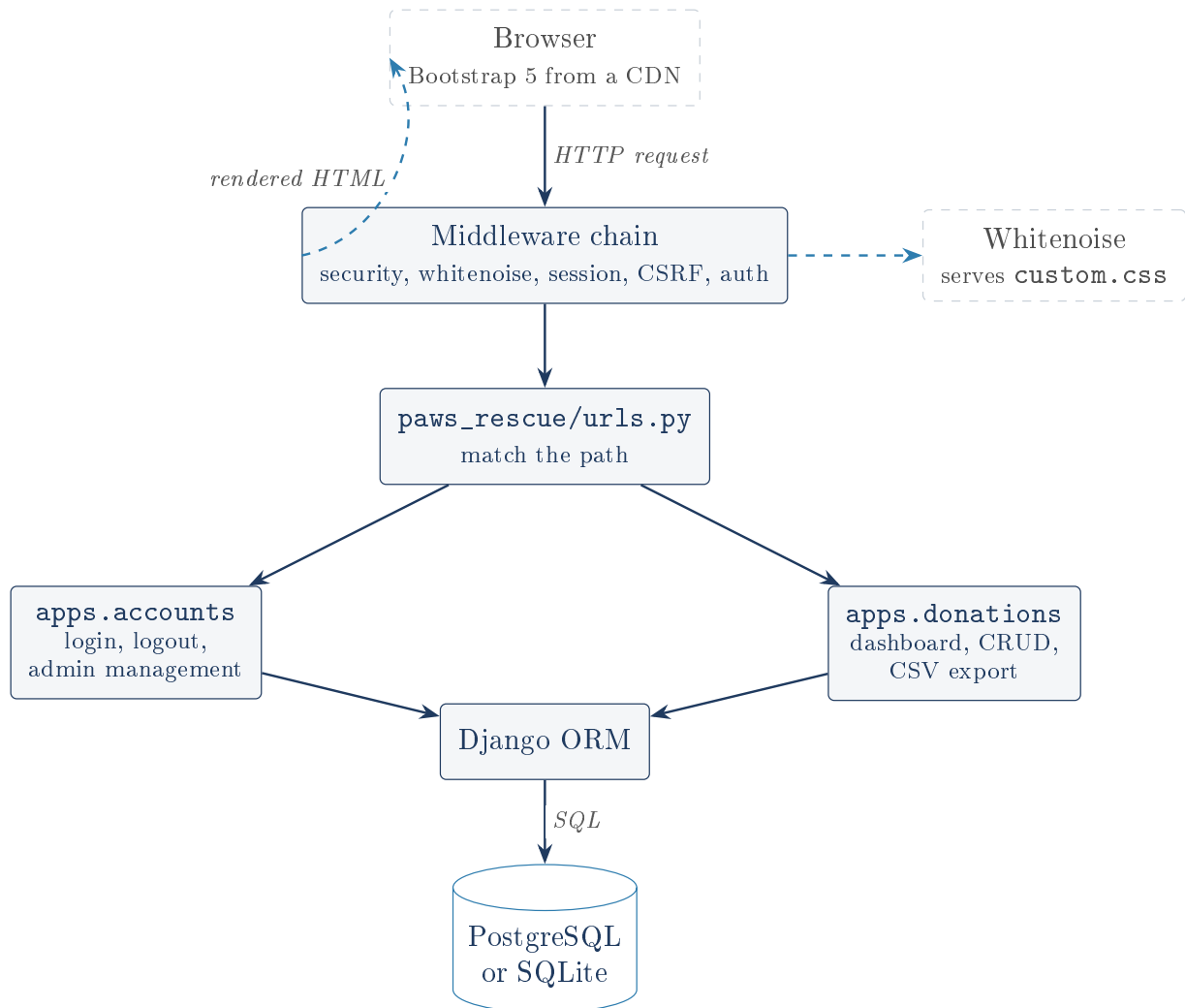


Figure 2: Request path. Everything is server-rendered: the browser receives finished HTML and runs no application JavaScript of the project’s own.

Jargon: Middleware

A stack of small components each request passes through on the way in, and each response passes back through on the way out. They handle cross-cutting concerns so that no individual view has to: attaching the logged-in user to the request, rejecting forged form submissions, compressing static files.

Jargon: CDN (Content Delivery Network)

A network of servers that hosts common files, here the Bootstrap CSS framework. The page’s HTML links out to `cdn.jsdelivr.net` rather than shipping Bootstrap in the repository. It keeps the repository small; it also means the styling silently breaks without an internet connection.

3.2 The two apps and what separates them

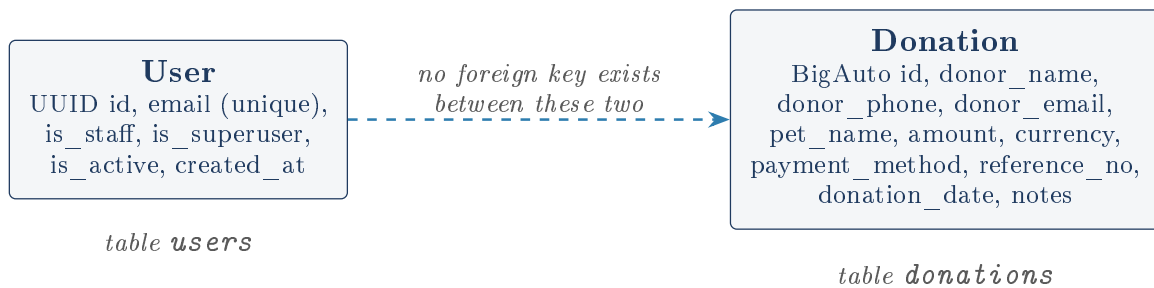


Figure 3: The complete data model. The dashed link is the relationship that is *absent*: nothing records which staff member entered a donation.

The missing relationship is the design’s biggest structural gap

A donation-tracking system for a charity has no audit trail. `Donation` carries no foreign key to `User`, so the database cannot answer “who entered this record” or “who deleted it”. Deletes are hard deletes: `donation.delete()` removes the row outright, with no tombstone and no history table. For a system whose entire purpose is tracking other people’s money, that is the first thing a reviewer will ask about. The project’s own README raises this too, under “What I’d do differently”; it is recorded here as confirmed against the schema in `apps/donations/migrations/0001_initial.py`, which contains no `ForeignKey` operation.

Jargon: Foreign key

A column in one table holding the identifier of a row in another table, which is how relational databases express “this donation was entered by that user”. The database then enforces that the referenced row actually exists.

4 The accounts app: who is allowed in

4.1 Why there is a custom user model at all

Django ships with a perfectly good `User` model. It has a `username` field, and that is the problem: this application wants people to log in with an email address and nothing else. Rather than carry a vestigial `username` column forever, the project defines its own user model.

```

1 class User(AbstractBaseUser, PermissionsMixin):
2     id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
3     email = models.EmailField(unique=True, max_length=255)
4     is_staff = models.BooleanField(default=False)
5     is_superuser = models.BooleanField(default=False)
6     is_active = models.BooleanField(default=True)
7     created_at = models.DateTimeField(default=timezone.now)
8     updated_at = models.DateTimeField(auto_now=True)
9
10    objects = UserManager()
11
12    USERNAME_FIELD = 'email'
13    REQUIRED_FIELDS = []
  
```

Listing 1: `apps/accounts/models.py`, the model itself

Three decisions are packed into those lines.

`AbstractBaseUser` plus `PermissionsMixin`. `AbstractBaseUser` gives password hashing and session handling but no fields beyond `password` and `last_login`, leaving the identity fields to you.

PermissionsMixin adds the `is_superuser` flag and the group and permission relationships Django’s own admin site expects.

`USERNAME_FIELD = 'email'` tells Django which field is the login identifier. Everything downstream (the `authenticate()` function, the admin site) reads this attribute rather than hard-coding “username”.

A UUID primary key.

Jargon: Primary key, and UUID

The primary key is the column uniquely identifying each row. The default is a *sequential integer*: 1, 2, 3. A UUID (Universally Unique Identifier) is instead a random 128-bit value such as 09c139f5-1275-4a83-92c0-5684f5a58e8b. Sequential integers leak information (user #3 can tell there are few users, and can guess that user #4 exists); UUIDs do not, and can be generated without asking the database.

The two primary key styles are inconsistent

User uses a UUID. Donation uses Django’s default sequential `BigAutoField`, so donation URLs read `/donations/7/edit/`. Neither choice is wrong on its own, but the project applies opposite reasoning to the two tables without recording why. If the argument for the UUID was non-enumerability, it applies at least as strongly to donation records, which are the sensitive data here. The honest reading, labelled **inferred** because no comment in the source states a rationale, is that the UUID came along with the hand-written user model and the default came along with the generated one.

4.2 UserManager: why dropping username forces this

Django’s default user manager has a `create_user(username, email, password)` signature. With no username field, that signature no longer fits, and every part of Django that calls it (notably `manage.py createsuperuser`) would break. The project supplies a replacement.

```

1 class UserManager(BaseUserManager):
2     def create_user(self, email, password=None, **extra_fields):
3         if not email:
4             raise ValueError('The Email field must be set')
5         email = self.normalize_email(email)
6         user = self.model(email=email, **extra_fields)
7         user.set_password(password)
8         user.save(using=self._db)
9         return user
10
11     def create_superuser(self, email, password=None, **extra_fields):
12         extra_fields.setdefault('is_staff', True)
13         extra_fields.setdefault('is_superuser', True)
14         if extra_fields.get('is_staff') is not True:
15             raise ValueError('Superuser must have is_staff=True.')
16         if extra_fields.get('is_superuser') is not True:
17             raise ValueError('Superuser must have is_superuser=True.')
18         return self.create_user(email, password, **extra_fields)

```

Listing 2: `apps/accounts/models.py`, the manager

Two details carry real weight. `normalize_email` lower-cases the domain part of the address, so `Admin@EXAMPLE.com` and `Admin@example.com` become the same string and the `unique=True` constraint actually bites. And `set_password` hashes the password rather than storing it:

Jargon: Password hashing

Storing a one-way scrambled form of a password instead of the password itself. Django's default here is PBKDF2 with SHA-256 and 600,000 iterations. Given the stored value you cannot practically recover the password, and the deliberate slowness makes guessing expensive. `set_password` does this; assigning to `user.password` directly would store plain text, which is exactly the mistake the manager exists to prevent.

`create_user` accepts `password=None`

Calling `create_user(email='x@y.com')` with no password is legal. Django's `set_password(None)` produces an unusable password rather than an empty one, so this does not create a login bypass, and the account simply cannot be logged into. It is worth knowing rather than fixing, because a reader will notice the default and wonder.

4.3 The login path

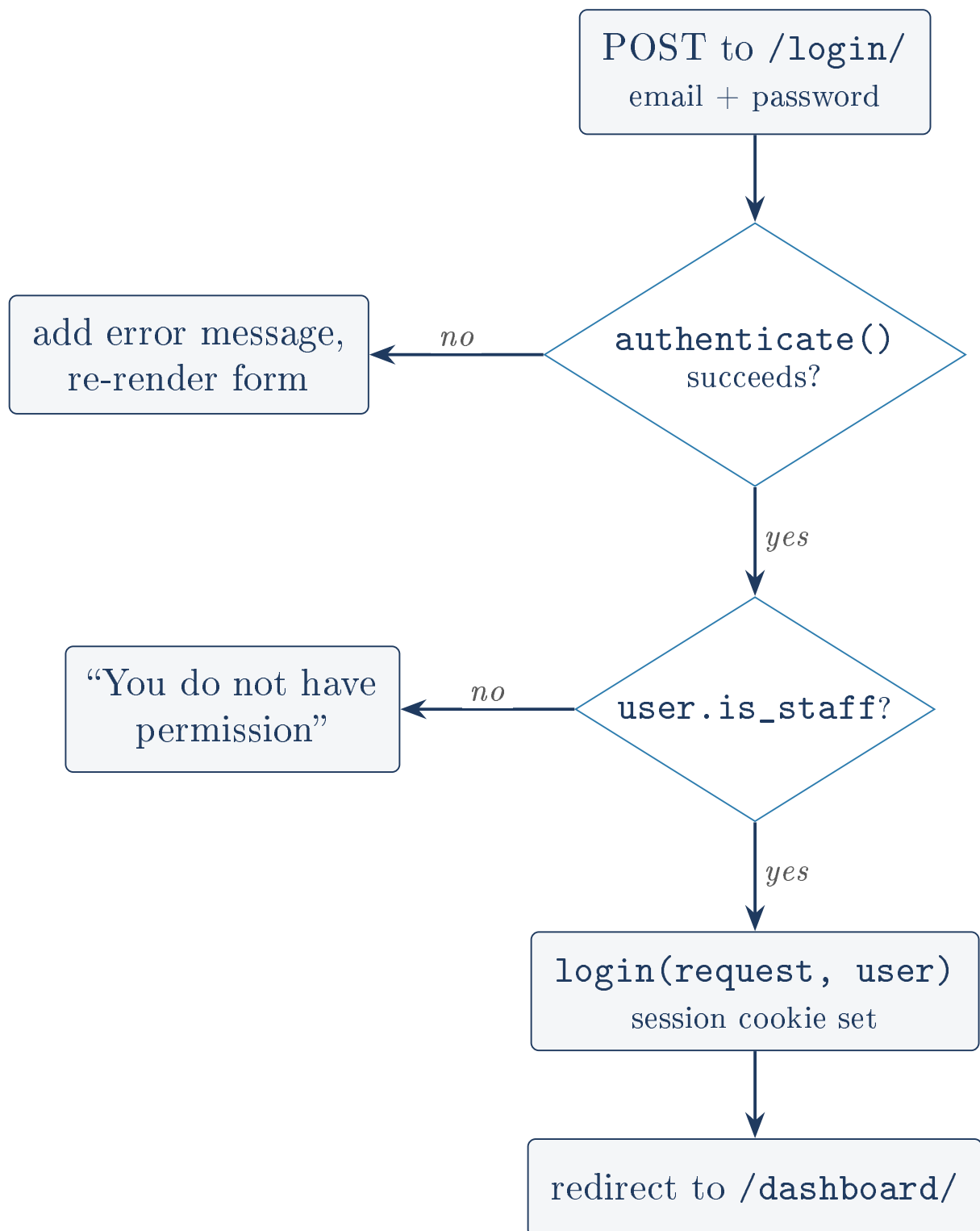


Figure 4: `login_view`. The `is_staff` gate is checked *after* authentication succeeds, and a non-staff user is never actually logged in.

`EmailAuthenticationForm` does the authentication inside its `clean()` method and stashes the result on `self.user_cache`, which `get_user()` returns. This mirrors how Django’s own `AuthenticationForm` works. Note the small awkwardness the custom user model forces:

```

1 self.user_cache = authenticate(
2     self.request,
3     username=email,      # the keyword is still 'username'
4     password=password
5 )

```

Listing 3: apps/accounts/forms.py, inside clean

Why username=email is correct, not a bug

`authenticate()` passes its keyword arguments through to the authentication backend, and Django’s `ModelBackend` looks up whatever field `USERNAME_FIELD` names using the value it received under the key `username`. The keyword is a fixed part of the interface; the value is an email. The same wrinkle shows up in the tests, which call `self.client.login(username='admin@test.com', ...)`.

4.4 Admin management, and where it is weaker than it looks

`admin_list_view` lists every `is_staff` user. `admin_create_view` creates one. `admin_delete_view` deletes one, refusing to delete the requester.

```

1 user_to_delete = get_object_or_404(User, id=user_id)
2
3 if user_to_delete.id == request.user.id:
4     messages.error(request, 'You cannot delete your own account.')
5     return redirect('admin_list')

```

Listing 4: apps/accounts/views.py, self-deletion guard

The guard is enforced in the view, not merely hidden in the template, which is the right way round: hiding a button does not stop anyone who can send an HTTP request. The template hides the button too, purely as a courtesy. This is covered by `test_cannot_delete_self`.

Every “admin” created through the web UI is a full superuser

`AdminCreationForm.save()` hard-codes both flags:

```

1 user.is_staff = True
2 user.is_superuser = True

```

There is no way to create a limited account through the interface, so the “admin management” feature has exactly one privilege level. The `PermissionsMixin`, groups and per-object permissions are all wired up and completely unused. For a rescue where a volunteer might reasonably be allowed to add donations but not to delete other logins, the distinction the model already supports is thrown away at the form layer.

The password validators do not apply to admin creation, verified by running it

`settings.py` configures the four standard `AUTH_PASSWORD_VALIDATORS` (similarity, minimum length, common-password, numeric-only). Those validators only run where something explicitly calls them, and `AdminCreationForm` never does: it subclasses `forms.ModelForm`, not Django’s `UserCreationForm`. Confirmed by executing the form directly against the project’s settings:

```

1 AdminCreationForm({'email': 'weak@example.com', 'password': '1'})
2 -> is_valid() == True, errors == {}

```

A one-character password is accepted for an account that is, per the finding above, a full superuser. The validators do run for `manage.py createsuperuser`, which is precisely why the

gap is easy to miss: the configuration looks present and correct at the top of `settings.py`. The fix is one line, calling `django.contrib.auth.password_validation.validate_password` inside a `clean_password` method.

Logging out is possible via GET

`logout_view` is decorated `@require_http_methods(["POST", "GET"])`. The navigation bar submits a proper POST form with a CSRF token, so normal use is fine, but the GET route remains open, which means any page anywhere can log the user out by embedding ``. This is a nuisance rather than a breach (no data is exposed and nothing is destroyed), and Django removed GET logout from its own `LogoutView` in version 4.1 for exactly this reason. Dropping "GET" from the list costs nothing, since the only caller already uses POST.

Jargon: CSRF (Cross-Site Request Forgery)

An attack where a page you are visiting quietly submits a request to a different site you happen to be logged into, riding on your session cookie. Django's defence is a secret token embedded in every form (`{% csrf_token %}`) and checked on submission; the attacking page cannot read the token, so its forged request fails. It only protects methods that change state, which is why leaving a state-changing action reachable by GET sidesteps the whole mechanism.

4.5 The `create_initial_admin` command

A chicken-and-egg problem: the app has no public sign-up, so a fresh deployment has no users and nobody can log in to create one. Django's interactive `manage.py createsuperuser` cannot help on a hosting platform with no terminal attached. The project adds a custom management command reading two environment variables.

```

1 email = os.getenv('INITIAL_ADMIN_EMAIL')
2 password = os.getenv('INITIAL_ADMIN_PASSWORD')
3
4 if not email or not password:
5     self.stdout.write(self.style.WARNING('... must be set ...'))
6     return
7
8 if User.objects.filter(email=email).exists():
9     self.stdout.write(self.style.WARNING(f'... already exists. Skipping.'))
10    return
11
12 User.objects.create_superuser(email=email, password=password)
```

Listing 5: `apps/accounts/management/commands/create_initial_admin.py`, the core

The existence check makes the command *idempotent*, which is what makes it safe to wire into a deploy script that runs on every release.

Jargon: Idempotent

Running it once and running it ten times produce the same result. Here, the second run finds the user already present and does nothing, rather than crashing on the unique-email constraint or resetting the password.

Both failure modes exit successfully and quietly

Missing variables print a warning and `return`. The command still exits with status code 0, meaning “success”. A deploy pipeline running `python manage.py create_initial_admin` and checking the exit code sees a green tick and moves on, and the operator discovers there is no admin account only when nobody can log in. Raising `CommandError` instead would make the deploy fail loudly, which is the behaviour you want.

5 The donations app: the actual subject matter

5.1 The Donation model

```

1 class Donation(models.Model):
2     PAYMENT_METHOD_CHOICES = [
3         ('cash', 'Cash'), ('bank', 'Bank Transfer'),
4         ('online', 'Online Payment'), ('other', 'Other'),
5     ]
6
7     donor_name = models.CharField(max_length=255)
8     donor_phone = models.CharField(max_length=50, blank=True, null=True)
9     donor_email = models.EmailField(blank=True, null=True)
10    pet_name = models.CharField(max_length=255, help_text='Name of the pet ...')
11    amount = models.DecimalField(max_digits=10, decimal_places=2,
12                                validators=[MinValueValidator(Decimal('0.00'))])
13    currency = models.CharField(max_length=10, default='PKR')
14    payment_method = models.CharField(max_length=20,
15                                    choices=PAYMENT_METHOD_CHOICES, default='cash')
16    reference_no = models.CharField(max_length=255, blank=True, null=True)
17    donation_date = models.DateTimeField(default=timezone.now)
18    notes = models.TextField(blank=True, null=True)
19    created_at = models.DateTimeField(auto_now_add=True)
20    updated_at = models.DateTimeField(auto_now=True)
21
22    class Meta:
23        db_table = 'donations'
24        ordering = ['-donation_date']

```

Listing 6: `apps/donations/models.py`, abridged

DecimalField, and why money is never a float

`amount` is a `DecimalField`, not a `FloatField`. Floating-point numbers store values in binary, and 0.1 has no exact binary representation, so `0.1 + 0.2` famously yields `0.30000000000000004`. Summing thousands of donations that way accumulates real error. A decimal stores the digits exactly. `max_digits=10`, `decimal_places=2` means up to eight digits before the point and exactly two after: the largest recordable donation is 99,999,999.99.

The difference between blank and null

They look redundant and are not. `null=True` is a *database* statement: this column may hold SQL NULL. `blank=True` is a *validation* statement: forms may leave this field empty. Django’s convention is to avoid `null=True` on text fields, because it creates two different ways to say “no value” (an empty string and NULL) that must then be handled separately forever. This project sets both on every optional text field, so `donor_phone` can be either `''` or `None`, which is why the CSV writer has to say `donation.donor_phone` or `''` on every such column.

`auto_now_add` stamps `created_at` once at insert; `auto_now` refreshes `updated_at` on every save.

`donation_date` is separate from both because when the money arrived and when somebody typed it in are different facts.

`Meta.ordering = ['-donation_date']` makes newest-first the default for every query that does not say otherwise. `db_table = 'donations'` overrides Django's generated name (`donations_donation`), which is a cosmetic preference.

currency is free text, and the consequence is worse than the README says

`currency` is a `CharField(max_length=10, default='PKR')` with no choices. Anything up to ten characters is accepted. The README's own self-assessment says typos such as "pkr" or "Rs" would "silently split the totals". Running the code shows the opposite, and the opposite is more dangerous. See the dedicated finding in §8.

5.2 Two forms, doing quite different jobs

`DonationForm` is a `ModelForm`: it derives its fields from the model, so the field list is declared once. Its only additions are Bootstrap CSS classes on each widget and two `clean` methods.

```

1 def clean_amount(self):
2     amount = self.cleaned_data.get('amount')
3     if amount and amount < 0:
4         raise forms.ValidationError('Amount must be greater than or equal to 0.')
5     return amount
6
7 def clean_donation_date(self):
8     """Set donation_date to now if it's empty."""
9     donation_date = self.cleaned_data.get('donation_date')
10    if not donation_date:
11        from django.utils import timezone
12        donation_date = timezone.now()
13    return donation_date

```

Listing 7: `apps/donations/forms.py`, the two cleaners

`clean_donation_date` implements “leave the field empty to mean now”. The model default (`default=timezone.now`) only applies when the field is omitted entirely; a form submitting an empty string is not an omission, so without this method an empty date box would be a validation error rather than a convenience.

`clean_amount` is a redundant second guard, and I checked which one actually fires

There are two negative-amount defences: `MinValueValidator` on the model field and `clean_amount` on the form. I instrumented the form and ran both paths against the project's real settings:

- With `clean_amount` present, submitting -100.00 is rejected with *its* message: “Amount must be greater than or equal to 0.”
- With `clean_amount` neutralised, -100.00 is *still* rejected, with the model validator's message: “Ensure this value is greater than or equal to 0.00.”

So the method is not dead code, but it is not load-bearing either: it only changes the wording. The reason both fire at all is worth knowing, because it is a genuine Django subtlety. Model-field validators are **not** copied onto the form field (I printed them: the form field carries only a `DecimalValidator`). They run later, in `ModelForm._post_clean()`, which calls `instance.full_clean()`. That is after every `clean_<field>` method. Hence `clean_amount` gets first refusal and its message wins.

The if amount and amount < 0 condition is subtly wrong, harmlessly

`Decimal('0.00')` is falsy in Python, so a zero amount short-circuits the `and` and skips the comparison entirely. It happens not to matter, because zero is not less than zero and should pass anyway. Verified: a 0.00 donation is accepted by the form. The condition should be `if amount is not None and amount < 0`; the same `is not None` care is taken correctly a few files away, in the amount filters in `dashboard_view`. The codebase knows the idiom and applies it inconsistently.

`DonationFilterForm` is a plain `forms.Form`, not tied to the model. Every one of its eight fields is `required=False`, which is what makes “filter by any combination of these” work. It exists to do type coercion and nothing else: it turns the raw query string `?amount_min=500&date_from=2026-01-01` into a validated `Decimal` and a real `date` object, or into an error.

5.3 dashboard_view: the heart of the application

This one function does filtering, sorting, aggregation, pagination and session-writing. It is the most consequential code in the project.

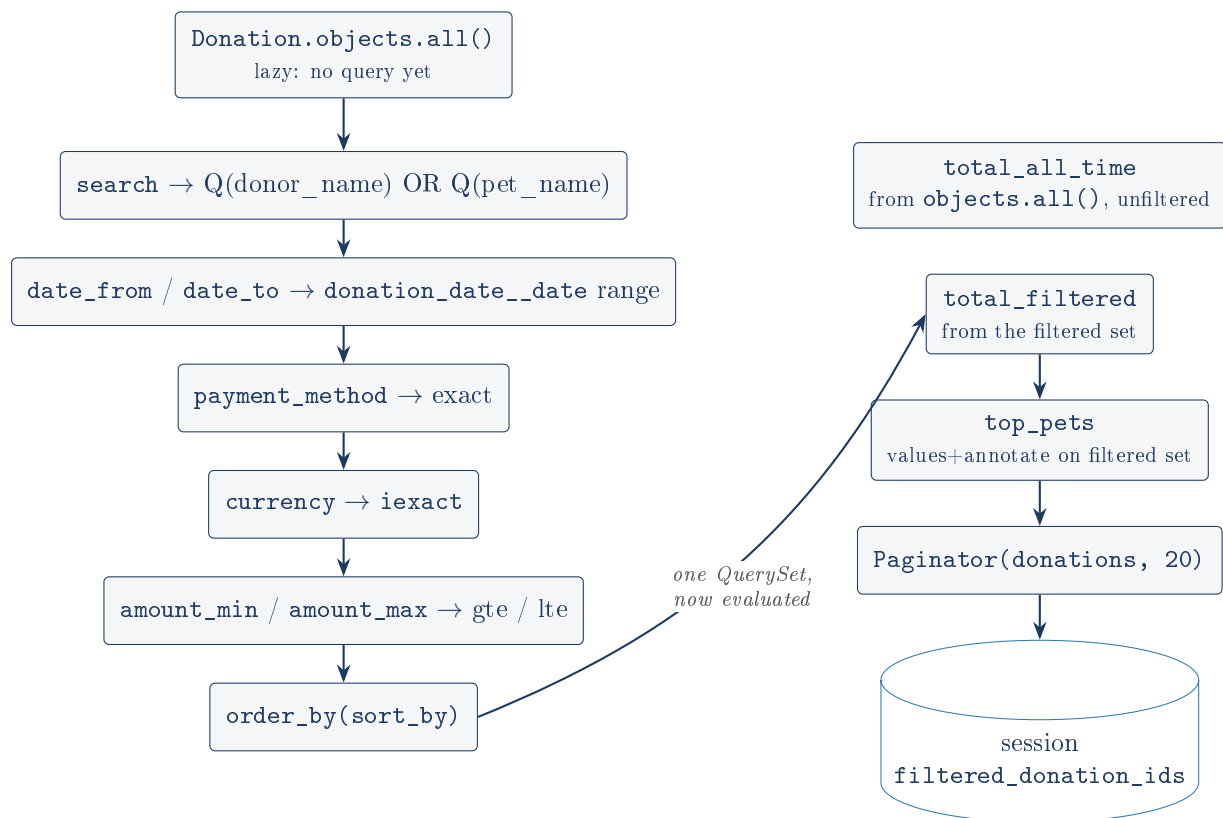


Figure 5: `dashboard_view`. Each filter is applied only if its cleaned value is present, so the seven are freely combinable. Note `total_all_time` deliberately branches off the *unfiltered* manager.

Why stacking filters like this is cheap

Each `donations = donations.filter(...)` line looks like it runs a query. None of them do. A `QuerySet` is lazy: `filter()` returns a new `QuerySet` with one more condition recorded, and the database is not touched until something iterates the results. All seven conditions collapse into a single `WHERE` clause in one SQL statement.

Jargon: Q object

Django’s way of expressing a condition that is not a simple AND. Keyword arguments to `filter()` are ANDed together, so there is no way to spell OR with them. A Q object wraps a condition so it can be combined with `|` (OR) and `&` (AND). The search box needs `Q(donor_name__icontains=s) | Q(pet_name__icontains=s)`: match either column.

Jargon: Field lookups: icontains, iexact, gte, lte

The double-underscore suffix on a filter keyword chooses the comparison. `icontains` is a case-insensitive substring match (SQL LIKE '%x%'), `iexact` a case-insensitive equality, `gte` and `lte` are “greater than or equal” and “less than or equal”. `donation_date__date__gte` chains two: extract the date part of the timestamp, then compare.

Why donation_date__date matters

`donation_date` is a timestamp, not a date. Filtering `donation_date__gte` against a date silently means midnight on that date, so a donation at 14:00 on the `date_to` day would be excluded from a range that visibly includes that day. The `__date` lookup extracts the calendar date first and compares dates to dates, which is what a user picking “to: 7 February” means.

The amount filters show the careful version of the idiom the form got wrong:

```
1 amount_min = filter_form.cleaned_data.get('amount_min')
2 if amount_min is not None:
3     donations = donations.filter(amount__gte=amount_min)
```

Listing 8: apps/donations/views.py, the amount bounds

`is not None` rather than a truthiness test, because `Decimal('0')` is falsy and “show me donations of at least 0” is a legitimate request that a bare `if amount_min:` would silently discard.

5.3.1 The aggregation, and a trap it avoids by luck

```
1 total_all_time = Donation.objects.aggregate(total=Sum('amount'))['total'] or 0
2 total_filtered = donations.aggregate(total=Sum('amount'))['total'] or 0
3
4 top_pets = (
5     donations
6     .values('pet_name')
7     .annotate(total_amount=Sum('amount'), donation_count=Count('id'))
8     .order_by('-total_amount')[:5]
9 )
```

Listing 9: apps/donations/views.py, analytics

Jargon: values plus annotate, i.e. GROUP BY

`.values('pet_name')` says “I care only about this column”. Following it with `.annotate(Sum('amount'))` makes Django group the rows by that column and compute the sum within each group. It is SQL’s `GROUP BY pet_name` with `SUM(amount)`, spelled in Python.

The README records that this panel once aggregated over `Donation.objects.all()` and therefore contradicted the filtered total displayed beside it. The current code aggregates over the filtered `donations` QuerySet, which is correct, and I confirmed the fix holds.

There is a well-known Django trap lurking here that the code does *not* fall into, and it is worth recording as verified rather than assumed. When a QuerySet carries an `order_by`, and this one

always does (`Meta.ordering`, plus an explicit `order_by(sort_by)` a few lines above), the ordering columns can be dragged into the `GROUP BY` clause, shattering each pet into one group per distinct timestamp. I tested it directly: two donations for the pet Max on different dates, totalling 400.

```

1 top_pets (default sort):      [{'pet_name': 'Max', 'total_amount': 400, 'donation_count': 2},
  ...]
2 top_pets (sort_by=-donation_date): [{'pet_name': 'Max', 'total_amount': 400, 'donation_count':
  2}, ...]
3 top_pets (sort_by=-amount):    [{'pet_name': 'Max', 'total_amount': 400, 'donation_count': 2},
  ...]
4
5 generated SQL:
6 SELECT "donations"."pet_name", CAST(SUM("donations"."amount") AS NUMERIC) AS "total_amount",
7       COUNT("donations"."id") AS "donation_count"
8 FROM "donations" GROUP BY "donations"."pet_name" ORDER BY 2 DESC LIMIT 5

```

Listing 10: Observed output, all three sort options

One row for Max, 400, and a `GROUP BY` on `pet_name` alone. The trailing sort on `-total_amount` is what saves it: an `order_by` *replaces* any previous ordering rather than adding to it, so the inherited date ordering is discarded before the SQL is built. The aggregation is correct. It is correct because a clause added for a different reason, sorting the panel by size, happens to clear the ordering that would have broken it. That is worth knowing if anyone ever removes that line.

The or 0 is load-bearing

Sum over zero rows returns `None`, not 0. Without `or 0` an empty database would render “None PKR” on the dashboard.

Jargon: Pagination

Splitting a long list across numbered pages. `Paginator(donations, 20)` slices the `QuerySet` into 20-row pages and, because the `QuerySet` is lazy, does it with `SQL LIMIT` and `OFFSET` rather than loading every donation into memory. `get_page` is the forgiving variant: a missing, non-numeric or out-of-range page number yields the first or last page instead of raising.

5.4 CSV export, and the two bugs in it

```

1 # in dashboard_view, on every render:
2 request.session['filtered_donation_ids'] = list(donations.values_list('id', flat=True))
3
4 # in export_csv_view:
5 filtered_ids = request.session.get('filtered_donation_ids', [])
6
7 if filtered_ids:
8     donations = Donation.objects.filter(id__in=filtered_ids).order_by('-donation_date')
9 else:
10    donations = Donation.objects.all().order_by('-donation_date')

```

Listing 11: `apps/donations/views.py`, the session handoff

The design problem it is solving is real. “Export CSV” is a plain link: a GET to the export URL, carrying no filter state at all, so the export view has no idea what the dashboard was showing. The chosen workaround: the dashboard writes the id of every matching donation into the session, and the export view reads them back.

Jargon: Session

Server-side storage tied to one browser via a cookie. The browser holds only an opaque key; the data (here, a list of donation ids) lives in the server’s `django_session` database table.

The README calls this “the weakest part of the design”. That is correct, and running it surfaces two concrete defects that the README does not name.

A filter matching nothing exports the entire database, verified by running it

if `filtered_ids`: cannot distinguish “the user has not loaded the dashboard, so export everything” from “the user’s filter matched zero rows, so export nothing”. Both are the empty list, and the empty list is falsy, so both take the `else` branch. I drove this through the real test client with two donations in the database:

```
1 GET /dashboard/?search=NoSuchDonorZZZ
2   -> dashboard displays "0 donation(s)"
3   -> session['filtered_donation_ids'] == []
4 GET /donations/export/
5   -> CSV data rows exported: 2
6   -> contains John? True | contains Jane? True
```

The user filters to nothing, sees an empty table reading “No donations found”, clicks Export CSV, and silently downloads every donor name, phone number and email address in the system. This is the most serious defect in the project: it is a real bug, it is triggered by an ordinary action, it fails in the unsafe direction, and it is invisible because the file downloads without complaint. The fix is one character of intent: if `filtered_ids` is not `None`: combined with storing `None` rather than never writing the key, or better, re-deriving the queryset from the query string.

The export is stale by construction

The session holds ids captured at the moment the dashboard was last rendered. Anything that happens afterwards is invisible to it. Verified: filter to donor “John” (session holds [1]), delete John’s donation in another window, then export. The `id__in` lookup simply finds nothing and the CSV comes back with 0 data rows, so this particular case degrades safely. A donation *added* after the render is likewise missing from an export the user believes reflects their filter. There is also no bound on growth: every dashboard render writes one integer per matching row into the session, and with `SESSION_SAVE_EVERY_REQUEST = True` in `settings.py` that row is rewritten to the database on every single request. At a few hundred donations this is invisible; it is still the wrong shape.

The fix the README already identifies

Re-apply `DonationFilterForm` to `request.GET` inside `export_csv_view` and rebuild the queryset from the query string, with the export link carrying the current filters. No session, no staleness, no growth, no empty-list ambiguity. It needs the filtering logic extracted into a shared helper, which the README also flags: right now `dashboard_view` applies the seven filters inline and nothing else can reuse them.

The writing itself is unremarkable and correct: `HttpResponse(content_type='text/csv')`, a `Content-Disposition` header naming the file `donations_YYYY-MM-DD.csv` to make the browser download rather than display it, and `csv.writer(..., quoting=csv.QUOTE_ALL)`.

Why `QUOTE_ALL` is the right call

CSV separates fields with commas, so a value containing a comma (“Bella, the tabby”, or any note with a comma in it) would otherwise split into two columns and corrupt every column after it. Quoting every field unconditionally makes that impossible.

`get_payment_method_display()` is worth pointing out: for any field with choices, Django gen-

erates a `get_<field>_display()` method returning the human label (“Bank Transfer”) rather than the stored value (“bank”). The CSV uses it; so does the dashboard template.

5.5 The three CRUD views

Jargon: CRUD

Create, Read, Update, Delete: the four basic operations on a record.

`donation_create_view` and `donation_update_view` are the standard Django form pattern, differing only by `instance=donation:` passing an existing object makes the same `ModelForm` an edit form rather than a create form. `donation_delete_view` shows a confirmation page on GET and deletes on POST, which is the correct division (a GET must never change state).

Form errors are pushed through the messages framework, twice over

Both write views do this on failure:

```
1 for field, errors in form.errors.items():
2     for error in errors:
3         messages.error(request, f'{field}: {error}')
```

The bound form is then re-rendered, and the template also displays each field’s errors inline. So a validation failure shows the user the same message twice, once in a red banner reading `donor_name: This field is required.` with the raw field name, and once beside the field. The banner version leaks the internal attribute name into the UI. Deleting the loop loses nothing.

`is_admin` is defined twice

The identical four-line function exists in both `apps/accounts/views.py` and `apps/donations/views.py`. Harmless today, and exactly how two definitions drift apart later.

6 Configuration: `settings.py`

6.1 Environment variables and the twelve-factor idea

```
1 load_dotenv()
2
3 SECRET_KEY = os.getenv('SECRET_KEY', 'django-insecure-dev-key-change-in-production')
4 DEBUG = os.getenv('DEBUG', 'False') == 'True'
5 ALLOWED_HOSTS = os.getenv('ALLOWED_HOSTS', 'localhost,127.0.0.1').split(',')
6 TESTING = 'test' in sys.argv
```

Listing 12: `paws_rescue/settings.py`, the top

Jargon: Environment variable, and `.env`

A named value handed to a program by the system that starts it, rather than written in its source. Secrets live there so they never enter version control. `python-dotenv` reads a local `.env` file into the environment for development convenience; `.env` is in `.gitignore`, and `.env.example` documents the required keys with placeholder values.

`DEBUG = os.getenv('DEBUG', 'False') == 'True'` is a string comparison because environment variables are always strings: the literal text `True` is the only value that enables debug mode, and

1, `true` or `yes` all leave it off. Defaulting to `False` is the right way round: a forgotten variable produces a secure app, not an insecure one.

The `SECRET_KEY` fallback silently produces a working, insecure app

`SECRET_KEY` falls back to a hard-coded string in the source. The key signs session cookies and CSRF tokens; anyone who knows it can forge either. The word `django-insecure` in the value is the only warning, it is committed to a public repository, and a deployment that forgets to set `SECRET_KEY` starts up perfectly happily using it. `DEBUG` was given a safe default; `SECRET_KEY` was not. Crashing on a missing key in production is the standard remedy.

6.2 The database switch

```
1 DATABASE_URL_ENV = os.getenv('DATABASE_URL')
2
3 if DATABASE_URL_ENV:
4     DATABASES = {'default': dj_database_url.config(
5         default=DATABASE_URL_ENV, conn_max_age=600, conn_health_checks=True)}
6 else:
7     DATABASES = {'default': {
8         'ENGINE': 'django.db.backends.sqlite3', 'NAME': BASE_DIR / 'db.sqlite3'}}
```

Listing 13: `paws_rescue/settings.py`, database selection

Set `DATABASE_URL` and the app talks to PostgreSQL; leave it empty and it uses a local SQLite file. This is what lets a new contributor run the project with no database server installed.

Jargon: SQLite versus PostgreSQL

SQLite is a database that is just a file on disk, with no server process. It is ideal for development and hopeless for a multi-user production deployment, where concurrent writes serialise behind a single lock and the file vanishes with the container's disk. PostgreSQL is a full server process built for exactly that.

A `conn_max_age` of 600 keeps a database connection alive for ten minutes rather than reconnecting on every request, since a TCP handshake plus authentication on each page load is real latency. Setting `conn_health_checks` to `True` is the necessary companion: it pings a reused connection before handing it over, so a connection the database closed in the meantime produces a reconnect, not an error page.

6.3 Static files and Whitenoise

Jargon: Static files

Files served exactly as they are, with no per-request computation: CSS, JavaScript, images. Django serves them itself in development and refuses to in production, on the grounds that a real web server does it better.

Jargon: Whitenoise

A library letting the Python application serve its own static files competently, with compression and cache headers, so a deployment needs no separate nginx. Its middleware sits second in the chain, immediately after Django's security middleware, so a request for a CSS file is answered and returned without waking the rest of the stack.

Jargon: collectstatic and manifest storage

`collectstatic` copies every static file from every app into one directory (`STATIC_ROOT`) for serving. `CompressedManifestStaticFilesStorage` goes further: it renames each file to include a hash of its contents (`custom.a1b2c3.css`) and writes a manifest mapping original names to hashed ones. This allows caching forever, since changed content means a changed filename. The cost is that `{% static 'css/custom.css' %}` now performs a manifest lookup and **raises an exception** if the entry is missing.

That cost is precisely what produced one of the project's documented battles:

```

1 if TESTING:
2     STORAGES = {"staticfiles": {
3         "BACKEND": "django.contrib.staticfiles.storage.StaticFilesStorage"}}
4 else:
5     STORAGES = {"staticfiles": {
6         "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage"}}
7
8 # ... and at the bottom:
9 if not DEBUG and not TESTING:
10     SECURE_SSL_REDIRECT = True
11     SESSION_COOKIE_SECURE = True
12     CSRF_COOKIE_SECURE = True
13     SECURE_BROWSER_XSS_FILTER = True
14     SECURE_CONTENT_TYPE_NOSNIFF = True
15     X_FRAME_OPTIONS = 'DENY'
```

Listing 14: `paws_rescue/settings.py`, the test carve-out

Why `TESTING = 'test'` in `sys.argv` has to exist

This is the single most instructive line in the settings file. `manage.py test` does *not* set `DEBUG=True`; it runs with whatever `DEBUG` the environment gives, which defaults to `False`. So a block guarded only by `if not DEBUG` applies during tests, and two things then break at once:

- `SECURE_SSL_REDIRECT` turns every test-client request into a 301 redirect to `https://`, so every assertion expecting a 200 fails.
- The manifest storage raises on the first `{% static %}` tag, because tests never run `collectstatic`.

Inspecting `sys.argv` for the string `test` is admittedly a hack, and it is the common one. The clean alternative is a separate settings module for tests.

Jargon: The production security flags, one at a time

`SECURE_SSL_REDIRECT` bounces plain HTTP to HTTPS. `SESSION_COOKIE_SECURE` and `CSRF_COOKIE_SECURE` tell the browser never to transmit those cookies over plain HTTP, so a network eavesdropper cannot lift a session. `SECURE_CONTENT_TYPE_NOSNIFF` stops browsers from guessing a file's type and executing an upload as script. `X_FRAME_OPTIONS = 'DENY'` forbids embedding the site in an iframe, defeating clickjacking. `SECURE_BROWSER_XSS_FILTER` sets a legacy header that modern browsers ignore entirely; it is harmless but does nothing.

The security block being conditional is a real fragility

These flags apply only when `DEBUG` is false and tests are not running. The condition works, but it means the production security posture is determined by two unrelated inferences (an environment string, and the contents of `sys.argv`) rather than declared. The README

names the fix, splitting settings into base, dev and prod modules, and this is the honest assessment of why: today, anyone who runs the app with `DEBUG=True` on a public host loses all six protections silently and simultaneously.

6.4 Sessions

```
1 SESSION_COOKIE_AGE = 86400          # 1 day
2 SESSION_SAVE_EVERY_REQUEST = True
```

`SESSION_SAVE_EVERY_REQUEST` rewrites the session row on every request, which makes the one-day expiry a *sliding* window: an active user is never logged out mid-task, and an idle one is logged out a day later. The cost is a database write on every single request, which combines badly with the CSV export writing the whole id list into that session on every dashboard render.

7 Templates and the front end

`templates/base.html` defines the frame: Bootstrap 5 and Bootstrap Icons from a CDN, the project's `custom.css`, a navigation bar rendered only when `user.is_authenticated`, the message banners, and `{% block content %}`. Every other template begins `{% extends 'base.html' %}`.

Jargon: Template inheritance

A child template declares a parent and fills in named blocks the parent left open. The navigation bar and footer are written once. This is the same idea as a base class in object-oriented code.

Jargon: Auto-escaping

Django templates escape HTML by default: a donor named `<script>alert(1)</script>` renders as visible text, not executing script. This is what makes the dashboard safe against stored XSS (cross-site scripting) despite displaying user-entered strings everywhere, and it is on by default rather than by any decision in this project.

The pagination links do something worth noticing. To page through a filtered result set, the current filters must survive the click:

```
1 href="?page={{ page_obj.next_page_number }}{% for key, value in request.GET.items %}{% if key
   != 'page' %}&{{ key }}={{ value }}{% endif %}{% endfor %}"
```

Listing 15: `apps/donations/templates/donations/dashboard.html`

It re-emits every current query parameter except `page`. It works. It is also the kind of logic that belongs in a template tag rather than inline, and it is repeated four times in the file, once each for First, Previous, Next and Last.

The dashboard hard-codes PKR in three places

The two total cards and the top-pets panel all render ... PKR as literal text in the template, regardless of what currency the underlying rows actually hold. Combined with the free-text `currency` field, this produces the bug in §8. The table rows, by contrast, correctly print each donation's own `{{ donation.currency }}`, so a single page can show a row labelled USD sitting underneath a total labelled PKR that includes it.

8 The mixed-currency bug

Totals sum across every currency and label the result PKR, verified by running it

`Sum('amount')` adds the `amount` column with no regard for the `currency` column beside it, and the template then appends the literal string “PKR”. I drove this through the test client with one 100 PKR donation and one 100 USD donation:

```
1 total_all_time (100 PKR + 100 USD): 200
2 rendered page contains "200.00 PKR"? True
```

The dashboard states that the rescue has received 200.00 PKR. It has received 100 PKR and 100 USD. The number is meaningless and it is presented with total confidence, which is worse than an error message. The same applies to `total_filtered` and to every row of the `top-pets` panel.

The README’s own account of this is subtly wrong, and the truth is more dangerous

The README says, under “What I’d do differently”: “Validate `currency` against a choice list. It’s a free-text field defaulting to PKR, so typos (`'pkr'`, `'Rs'`) would silently split the totals.” Running it shows the reverse. Nothing splits. Everything merges:

```
1 distinct currency values stored: ['PKR', 'Rs', 'USD', 'pkr']
2 total shown on dashboard: one number, all four summed, labelled PKR
3 filter currency=PKR (uses __iexact) -> matches 2 rows: catches 'pkr', misses 'Rs'
```

The totals never split by currency because they never group by currency in the first place. `currency__iexact` means the *filter* also folds “pkr” into “PKR” correctly, so the case-typo scenario the README worries about is the one case already handled. The actual failure is the one the README does not mention: a genuinely different currency is added to the total as though it were rupees. Recording this because the README will be read by an interviewer as the author’s own assessment, and defending a self-assessment that the code contradicts is worse than having no self-assessment.

The correct fix is not only a choice list. Even with `currency` constrained to a clean enum, `Sum('amount')` across mixed currencies is still wrong. The dashboard needs to group by currency and show one total per currency, or convert at a recorded rate. The choice list makes the grouping reliable; it does not by itself make the arithmetic mean anything.

9 Testing

9.1 What is covered, and it genuinely passes

I created a clean virtual environment, installed the exact pinned dependencies from `requirements.txt`, and ran the suite rather than trusting the README’s count.

```
1 $ python manage.py test
2 Ran 11 tests in 12.828s
3
4 OK
```

Listing 16: Verified on this machine

Eleven tests, all passing, matching the README exactly. The dependency pins install cleanly (Django 4.2.11, pycopg2-binary 2.9.11, python-dotenv 1.0.1, gunicorn 21.2.0, whitenoise 6.6.0, dj-database-url 2.1.0).

Test	App	What it establishes
<code>test_unauthorized_user_redirected_to_login</code>	accounts	the login gate exists
<code>test_admin_can_login</code>	accounts	valid credentials redirect
<code>test_admin_list_requires_login</code>	accounts	protected before, reachable after
<code>test_cannot_delete_self</code>	accounts	self-deletion guard holds
<code>test_admin_can_access_dashboard</code>	donations	dashboard renders for staff
<code>test_admin_can_create_donation</code>	donations	create writes the right row
<code>test_amount_validation</code>	donations	a negative amount is refused
<code>test_csv_export_returns_csv_response</code>	donations	headers and content
<code>test_search_by_donor_name</code>	donations	search matches donor
<code>test_search_by_pet_name</code>	donations	search matches pet
<code>test_filter_by_payment_method</code>	donations	method filter runs

Table 1: The eleven tests. All eleven pass; the coverage claim is accurate.

Jargon: Test database

Django’s test runner creates a throwaway database, runs every migration into it, and destroys it afterwards. Each test method runs inside a transaction that is rolled back at the end, so tests cannot see each other’s data. This is why `setUp` can create the same admin for every test without colliding.

9.2 Where the suite is weaker than its pass rate suggests

Every assertion is a substring match against the whole HTML page

`assertContains(response, 'John Doe')` searches the entire rendered response, navigation bar, form labels, footer and all. The project already has a scar from this: the README records that `assertNotContains(response, 'Max')` failed on a page with no donation for the pet Max, because the filter form’s “Max Amount” label contained the substring. The resolution was to rename the label to “Amount To”. I confirmed the current labels in `apps/donations/forms.py` are ‘Amount From’ and ‘Amount To’, so the test passes today. The fix is worth being honest about, because it is backwards: the *user interface* was changed to suit the test’s matching strategy. The test is still fragile, and the next fixture named after any word appearing in the site’s chrome revives the same failure. Asserting on `response.context['page_obj']` would test the actual behaviour and could not collide with page text.

Three tests assert less than their names claim

- `test_filter_by_payment_method` filters to cash and asserts only that “John Doe” (the cash donor) is present. It never asserts that Jane Smith, the online donor, is *absent*. A completely broken filter that ignored the parameter and returned every row would pass this test.
- `test_amount_validation` posts a negative amount and asserts only that no row was created. It never checks the status code or that an error was shown, so a view that crashed with a 500 would also pass.
- `test_csv_export_returns_csv_response` calls the export with no filter applied, which is the branch that works. Neither this test nor any other exercises the export after a filter, which is why the empty-filter bug in §10 survives a green suite.

The two most consequential bugs are both in untested territory

The suite passes completely and both bugs I found are real. Nothing tests the export with a filter applied, and nothing tests a total with more than one currency present. This is the useful lesson from the project: eleven green tests bought real confidence about authentication and CRUD, and no confidence at all about the two features most likely to produce a wrong number in front of a user.

10 Honest findings, collected

Ordered by how much they would matter to a reviewer.

1. A zero-match filter exports the whole database (verified)

`export_csv_view`'s `if filtered_ids:` cannot tell an empty result from an absent session key, so filtering to nothing and clicking Export CSV downloads every record, including every donor phone number and email address. Confirmed by execution. This is a live bug, not a design preference.

2. Mixed-currency totals are summed and labelled PKR (verified)

100 PKR plus 100 USD renders as “200.00 PKR”. Confirmed by execution. The README's description of this area (“typos would silently split the totals”) is the opposite of what the code does.

3. `db.sqlite3` is committed, containing a password hash and live sessions

The database file is tracked in git and `.gitignore` does not exclude it. It was added deliberately (commit `a49871e`, “Add local dev DB from the screenshot verification run”). Its contents, read directly:

```
1 users:          1 row -> admin@example.com, is_staff=1, is_superuser=1,
2                  password = pbkdf2_sha256$600000$... (a real hash)
3 django_session: 2 rows -> session keys, expired 2026-07-08
4 donations:      0 rows
```

The exposure is limited and worth stating precisely rather than alarmingly: the email is the placeholder from `.env.example`, the donations table is empty, the sessions expired, and PBKDF2 at 600,000 iterations is not casually reversible. So this is not an emergency. It is still a committed credential hash and a bad habit: the file is build output, it will drift with every local run, it will produce meaningless binary diffs, and the next person who commits it after entering real donor data publishes real donor data. `db.sqlite3` belongs in `.gitignore` and the file should be removed from tracking; if the screenshot needs data, a fixture or a seed command is the tool.

4. Web-created admins are unvalidated superusers

A one-character password is accepted (verified), and every account created through the UI gets `is_superuser = True`. The two compound: the weakest possible password on the most privileged possible account, in an app whose whole security model is “there is a login”.

5. No audit trail

`Donation` has no foreign key to `User`. Nothing records who entered or deleted a record, and deletes are hard deletes. For a donation tracker this is the most significant *structural* gap, as opposed to the bugs above.

6. SECRET_KEY has a working insecure default

A deployment that forgets the variable runs happily on a key published in the repository.

7. Session-based export state

Unbounded growth (one integer per matching row, per dashboard render, rewritten to the database on every request thanks to `SESSION_SAVE_EVERY_REQUEST`) and staleness by construction. The README identifies this one accurately.

8. Missing indexes

`donation_date` drives the default ordering and `pet_name` drives both the search and the top-pets grouping. Neither is indexed, so both are full table scans. At this data volume it is invisible, and it is the first thing that would bite at scale. The README flags this.

9. Smaller things, listed for completeness

- `UserCreationForm` is imported in `apps/accounts/forms.py` and never used. Dead import.
- `is_admin` is defined identically in two files.
- `clean_amount` duplicates the model's `MinValueValidator` and only changes the error wording (verified); its `if amount and ...` test should be `if amount is not None and ...`.
- `logout_view` still permits GET.
- `django.contrib.admin` is installed and routed at `/admin/`, but neither app has an `admin.py`, so nothing is registered and the built-in admin shows only groups and permissions. It is dead weight, or an unfinished intention.
- Form errors are surfaced twice, once with raw field names in a banner.
- `.env.example` ships `DEBUG=True` while `settings.py` defaults to `False`. Both defensible, mildly contradictory.
- `SECURE_BROWSER_XSS_FILTER` sets a header every modern browser ignores.
- Bootstrap loads from a CDN, so the UI is unstyled offline.

11 What is genuinely good here

An honest document says this part too, and it is not padding.

- **The custom user model was done on day one.** `AUTH_USER_MODEL` is almost impossible to change after the first migration. Getting it right at the start is the single highest-leverage decision in a new Django project, and the README's "What I learned" shows the author knows why.
- **Money is a Decimal.** Common enough to get wrong that getting it right deserves the credit.
- **is not None on the amount filters.** A precise, deliberate guard against a real edge case, showing the author was thinking about falsy zero.

- **The TESTING flag is a genuine debugging win.** Diagnosing why every test suddenly 301-redirects to https requires understanding that `manage.py test` does not set `DEBUG`. That is a non-obvious chain and the README reconstructs it correctly.
- **The top-pets fix.** Noticing that a panel disagreed with the total beside it, and tracing it to aggregating over the unfiltered manager, is exactly the kind of bug most people ship.
- **The self-deletion guard is enforced server-side**, not just hidden in the template.
- **The README volunteers its own weaknesses.** Most of §10 items 5, 7 and 8 are the author's own findings. That is rarer than it should be, and it is worth defending as a deliberate practice.

12 If this project continued

In the order a maintainer should actually take them:

1. Fix the empty-filter export. It is a live data-exposure bug and a one-line change. Add the test that catches it.
2. Group totals by currency, or constrain `currency` and then group. Stop printing a number that is not true.
3. Untrack `db.sqlite3` and add it to `.gitignore`.
4. Validate passwords in `AdminCreationForm`, and stop making every admin a superuser.
5. Add `entered_by = ForeignKey(User, on_delete=PROTECT)` to `Donation` and stop hard-deleting.
6. Re-derive the export queryset from the query string and delete the session handoff, which retires findings 1 and 7 together.
7. Extract the filter logic into a shared helper or adopt `django-filter`.
8. Make `SECRET_KEY` mandatory in production.
9. Index `donation_date` and `pet_name`.
10. Split settings into base, dev and prod, retiring the `sys.argv` hack.

The one-sentence summary

A small, conventional, competently built Django CRUD application that gets several non-obvious Django decisions right (custom user model up front, Decimal money, lazy QuerySet filtering, a real reason for its test-mode flag) and carries two real bugs in the two places nobody tested: an empty filter exports everything, and mixed currencies are added together and labelled PKR.

Every claim in this document was checked against the source. Where a claim is marked verified, it was established by executing the code: the test suite, the form validation probes, the aggregation SQL, the export behaviour, and the contents of the committed database. The single item labelled inferred is the rationale for the mixed primary-key styles, which no comment in the source explains.